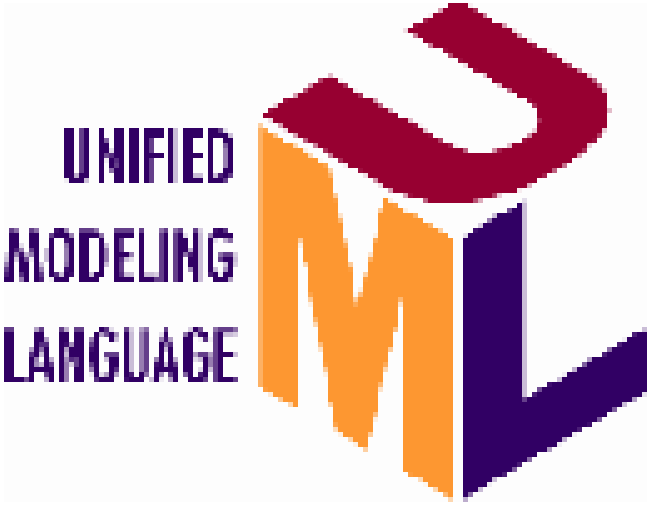
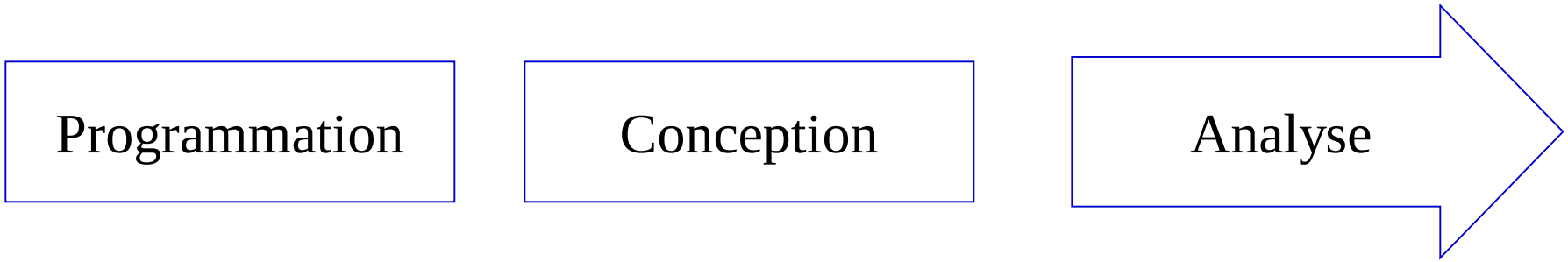


UML – Langage unifié pour la modélisation objet

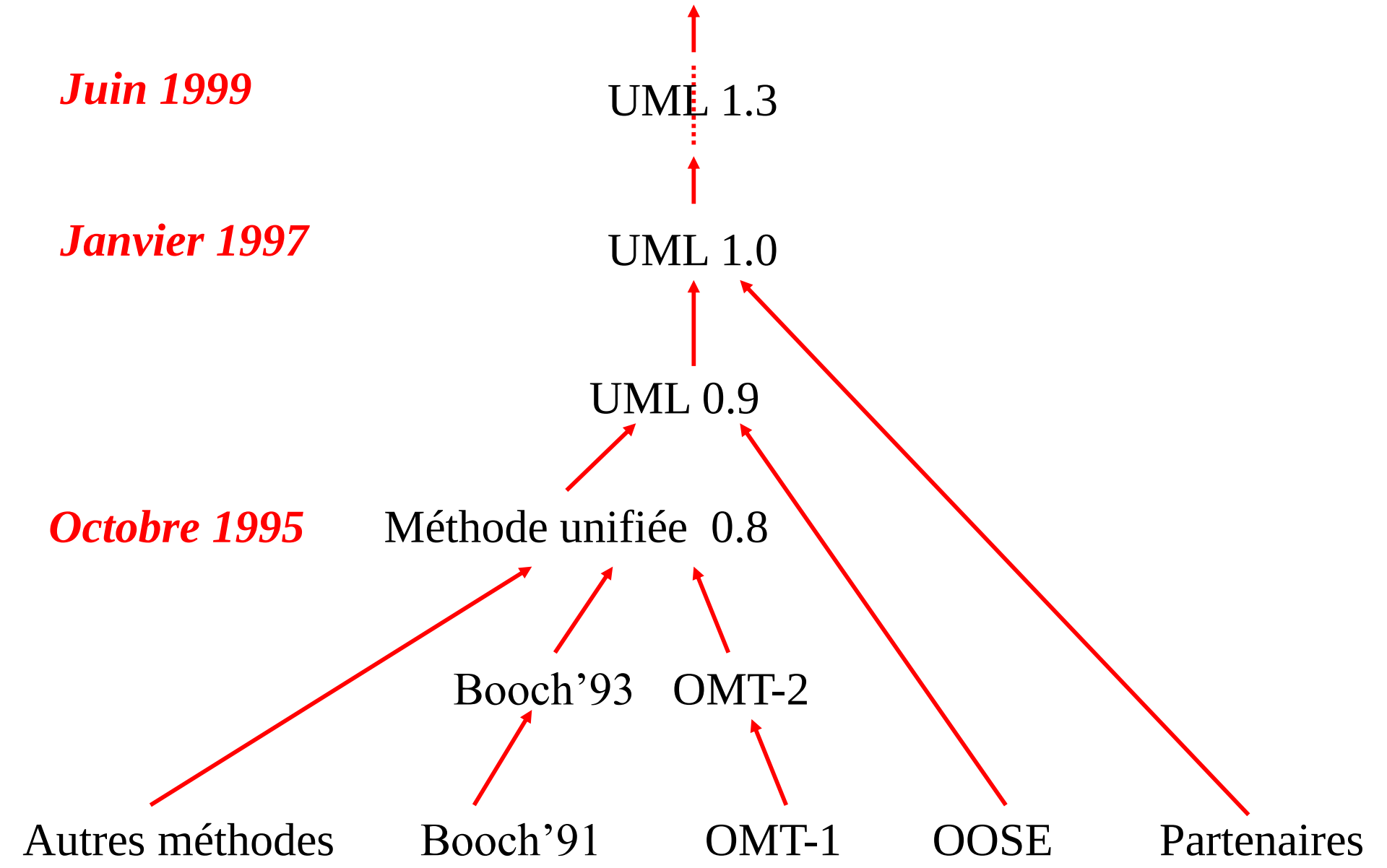
Évolution des méthodes



Booch, Jacobson et Rumbaugh se fixent 4 objectifs:

- *représenter des systèmes entiers (au-delà du seul logiciel) par des concepts objets,*
- *établir un couplage explicite entre les concepts et les artefacts exécutables,*
- *prendre en compte les facteurs d'échelle inhérents aux systèmes complexes et critiques,*
- *créer un langage de modélisation utilisable à la fois par les humains et les machines.*

Principales étapes de la définition d'UML



L'unification

Les créateurs d'UML insistent tout particulièrement sur le fait que la notation UML est un langage de modélisation objet et non pas une méthode objet.

UML n'est pas une notation propriétaire; elle est accessible à tous et les fabricants d'outils ainsi que les entreprises de formation peuvent librement en faire usage.

En français, UML pourrait se traduire par langage unifié pour la modélisation objet, mais il est plus probable qu'UML se traduise plutôt par notation unifiée, voire notation UML…

Modèle et modélisation

La modélisation n'est qu'une représentation d'un système réel quelle qu'en soit sa forme : physique, graphique, mathématique, verbale ou mentale. Cette représentation intelligible est indispensable pour assurer la compréhension de systèmes naturels complexes.

Modèles d'UML

- *Le modèle des cas d'utilisation qui décrit les besoins des utilisateurs*
- *Le modèle des classes qui capture la structure statique*
- *Le modèle des états qui exprime le comportement dynamique des objets*
- *Le modèle d'interaction qui représente les scénarios et les flots de messages*
- *Le modèle de déploiement qui précise la répartition des processus*

Diagrammes et vues externes

Les modèles sont regardés et manipulés par les utilisateurs au moyen de vues graphiques, véritables projections au travers des éléments de modélisation contenus par un ou plusieurs modèles. De nombreuses vues peuvent être construites à partir des modèles de base; elles peuvent montrer tout ou partie des modèles. A chaque vue correspondent un ou plusieurs diagrammes.

Les diagrammes d'UML

- *Vues statiques*
  - *Les diagrammes de classes*
  - *Les diagrammes d'objets*
  - *Les diagrammes de cas d'utilisation*
  - *Les diagrammes de composants*
  - *Les diagrammes de déploiement*
- *Vues dynamiques*
  - *Les diagrammes de séquence*
  - *Les diagrammes de communication*
  - *Les diagrammes d'états-transition*
  - *Les diagrammes d'activités*

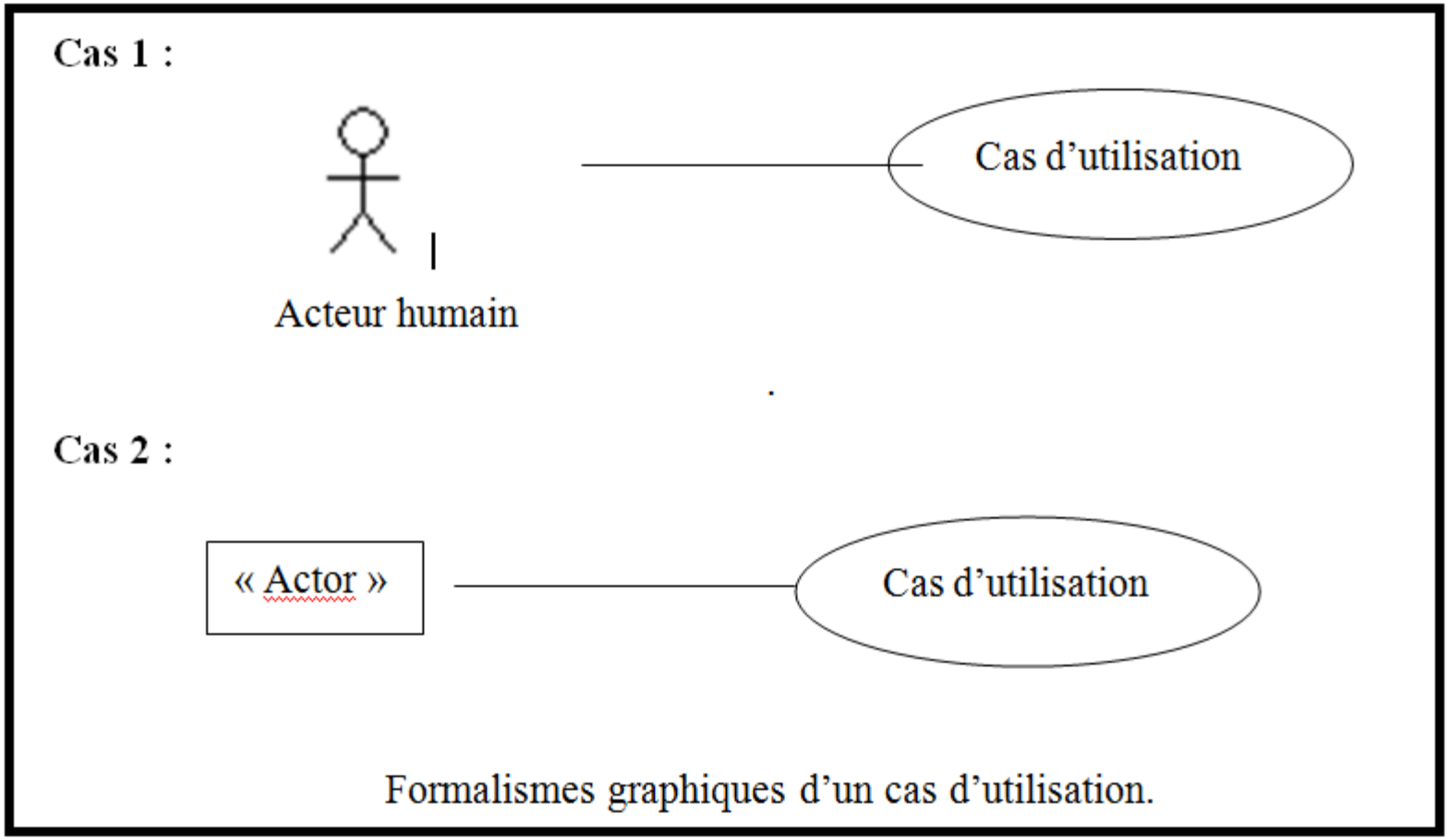
Le diagramme de cas d'utilisation

Objectifs des cas d'utilisation:

- Ils décrivent le comportement du système du point de vue utilisateur sous la forme d'actions et de réactions.
- Ils servent à structurer **les besoins des utilisateurs** et les objectifs correspondants du système.
- Un cas d'utilisation est une manière spécifique d'utiliser un système. C'est l'image d'**une fonctionnalité du système, déclenchée en réponse à la stimulation d'un acteur externe et produit un résultat observable par cet acteur.** Ils apportent une solution au problème de la détermination et de la compréhension des besoins.
- Ils permettent aux utilisateurs de structurer leurs besoins ; ils les obligent à définir la manière dont ils voudraient interagir avec le système, à préciser quelles informations ils entendent échanger et à décrire ce qui doit être fait pour obtenir le résultat escompté.

Le diagramme de cas d'utilisation

- Eléments constitutifs des cas d'utilisation



Formalismes graphiques d'un cas d'utilisation.

Acteur

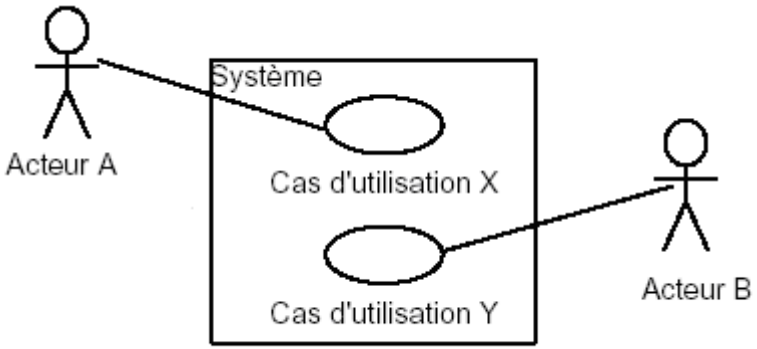
Entité externe qui agit sur le système et échange des informations avec lui ; le terme acteur ne désigne pas seulement les utilisateurs humains mais également les autres systèmes.

- Les acteurs principaux : les personnes qui utilisent les fonctions principales du système. Un acteur principal est celui qui bénéficie de l'utilisation du système.
- Les acteurs secondaires : les personnes qui effectuent des tâches administratives ou de maintenance.
- Le matériel externe : les dispositifs matériels incontournables qui font partie du domaine de l'application et qui doivent être utilisés.
- Les autres systèmes : les systèmes avec lesquels le système développé doit interagir.
-

Cas d'utilisation (Use-case)

Ensemble d'actions réalisées par le système en réponse à une action d'un acteur. L'ensemble des cas d'utilisation décrit les objectifs du système.

- Les cas d'utilisation peuvent être structurés.
- Les cas d'utilisation peuvent être organisés en paquetages.



Formalisme graphique du diagramme de cas d'utilisation préliminaire.

DESCRIPTION DES CAS D'UTILISATION

• Un cas d'utilisation est une abstraction de plusieurs chemins d'exécution d'une utilisation du système. Pour décrire un cas d'utilisation, il nous faut décrire un maximum de chemins d'exécutions possibles pour la séquence d'actions correspondant à ce cas. On étudiera donc un certain nombre de scénarios d'un cas d'utilisation.

• Un scénario est donc une instance d'un cas d'utilisation. On essaye de chercher le ou les scénarios nominaux et plusieurs scénarios alternatifs (qui se terminent de façon normale) ou d'exception (qui se terminent en échec). Nous aurons donc deux niveaux de description :

Le diagramme de cas d'utilisation

•Description générale d'un cas d'utilisation reprenant les divers chemins pouvant être réunis en un même cas. •
•Description des scénarios alternatifs et d'exception les plus pertinents.

Exemple : Scénario nominal « Retirer de l'argent ».
• Le client de la banque s'identifie.
• Le système lui pose les différents comptes sur lesquels il peut effectuer un retrait.
• Le client choisit le compte à débiter et spécifie le montant du retrait.
• Le système vérifie si le retrait est autorisé, si oui il déduit le montant du compte et délivre l'argent, sinon il renvoie un message de rejet de l'opération.

Les scénarios permettent de mieux comprendre le fonctionnement du système, en particulier ils sont utiles dans la description des cas d'erreurs et des cas limites d'utilisation du système. Ils permettent d'expérimenter les exécutions du système et sont donc très utiles pour les phases de tests et de maintenance.
Un cas d'utilisation peut donner lieu à un ensemble de scénarios.

STRUCTURATION DES CAS D'UTILISATION

Il est utile de restructurer l'ensemble des cas d'utilisation que l'on a fait apparaître afin de rechercher les comportements partagés, les cas particuliers et les généralisations.

Les relations possibles entre cas d'utilisation :

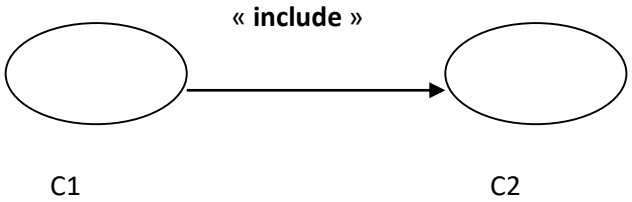
- Relation d'inclusion :

  Formalisée par la dépendance « **include** ».
- Relation d'extension:

  Formalisée par la dépendance « **extend** ».
- Relation de généralisation/spécialisation.

LA RELATION D'INCLUSION

Lors de la description des cas d'utilisation, il apparaît qu'il existe des sous ensembles communs à plusieurs cas d'utilisations, il convient donc de factoriser ces fonctionnalités en créant de nouveaux cas d'utilisation qui seront utilisés par les cas d'utilisation qui les avaient en commun.



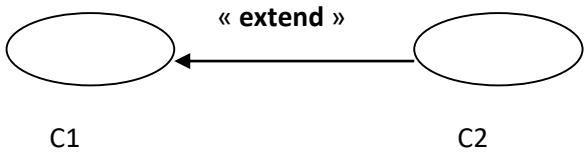
Formalisme graphique de la relation d'inclusion.

C1 dépend de C2.

C1 utilise C2 ⇒ **toute activation de C1 entraîne une activation de C2.**

LA RELATION D'EXTENSION

La relation « extend » permet d'étendre les interactions et donc les fonctions décrites par les interactions. Le cas d'utilisation de base peut fonctionner tout seul, mais il peut également être complété par un autre, sous certaines conditions, et uniquement à certains points de son flot d'événements. On utilise principalement cette relation pour séparer le comportement optionnel du comportement obligatoire.



Formalisme graphique de la relation d'extension.

C2 étend C1 ⇒ C2 est une façon particulière de réaliser C1.
Une instance de C1 va engendrer une instance de C2 et l'exécuter sous certaines conditions.
C2 connaît C1 et non l'inverse.
C2 n'existe pas tout seul et C1 existe sans C2.

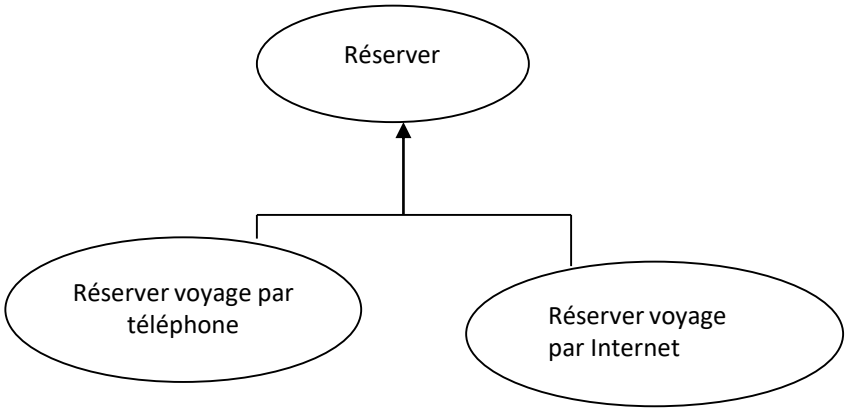
LA RELATION D'EXTENSION

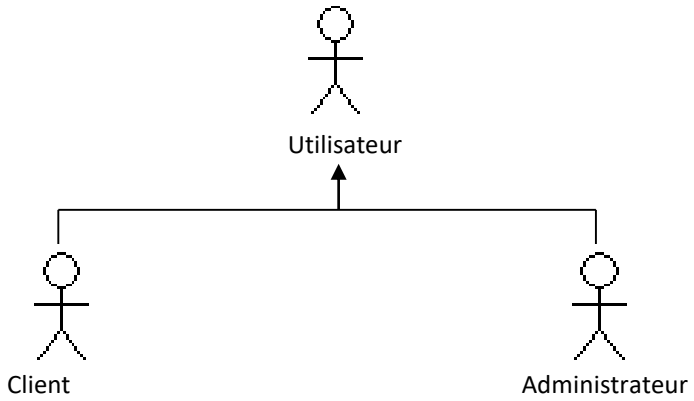
Point d'extension :

L'extension peut intervenir à un point précis du cas étendu. Ce point s'appelle le point d'extension. Il porte un nom, qui figure dans un compartiment du cas étendu sous la rubrique
point d'extension, et est éventuellement associé à une contrainte indiquant le moment où l'extension intervient. Une extension est souvent soumise à condition

LA RELATION DE GENERALISATION/ SPECIALISATION

Cette relation est à prendre au sens classique de généralisation/spécialisation. La généralisation peut être vue aussi comme un polymorphisme de cas.
On peut généraliser les acteurs et les cas d'utilisation.





PARTITIONNEMENT DES CAS D'UTILISATION

La notion de paquetage est une technique qui permet de mettre en œuvre le partitionnement hiérarchique des modèles tout en préservant la cohérence de l'ensemble.
Un paquetage est un ensemble d'éléments de modélisation.

Plusieurs stratégies sont possibles : procéder au regroupement par acteur ou par domaine fonctionnel.
Le découpage des cas d'utilisation est un découpage macroscopique et c'est le premier découpage du modèle.

Etude de cas : Etude d'un guichet automatique bancaire

Cette étude de cas concerne un système simplifié de Guichet Automatique de Banque(GAB).

Le GAB offre les services suivants :

1. Distribution d'argent à tout porteur de carte de crédit, via un lecteur de carte et un distributeur de billets.
2. Consultation de solde de compte, dépôt de chèques, pour les clients porteurs d'une carte de crédit de la banque adossée au GAB.
3. Toutes les transactions sont sécurisées.
4. Il est parfois nécessaire de recharger le distributeur, etc.

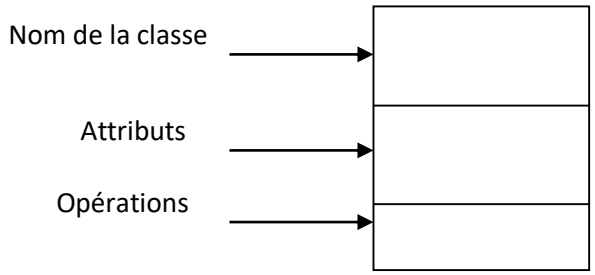
Le Diagramme de classes

Diagramme de classes

- Les diagrammes de classes expriment de manière générale la structure statique d'un système, en termes de classes et de relations entre ces classes. Une classe décrit un ensemble d'objets, une association décrit un ensemble de liens.
- Le diagramme de classes est alors un modèle permettant de décrire de manière abstraite et générale les liens entre objets.

Les classes

- Une classe est une description d'un ensemble d'objets partageant la même sémantique, ainsi que les mêmes attributs, opérations et relations.
- Une classe correspond à un concept global d'information et se compose d'un ensemble d'informations élémentaires, appelées attributs de la classe qui servent à la décrire.
- Une opération est une fonctionnalité assurée par la classe.
- Une classe peut être représentée comme suit :



Représentation graphique d'une classe.

Diagramme de classes

La déclaration d'un attribut a la syntaxe suivante :

Nom_attribut : Type_attribut = Valeur_Initiale.

La déclaration d'une opération a la syntaxe suivante :

Nom_opération(Nom_argument :Type_argument=Valeur_par_defaut…) : Type_Retourné.

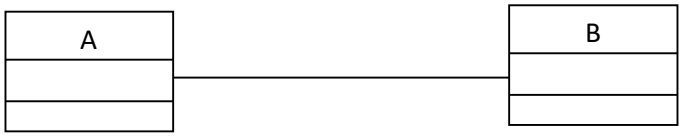
UML définit 3 niveaux de visibilité pour les attributs et les opérations:

- public : qui rend l'élément visible à tous les clients de la classe.
- protégé : qui rend l'élément visible aux sous-classes de la classe.
- privé : qui rend l'élément visible seulement à la classe.

L'information de visibilité ne figure pas toujours de manière explicite dans les diagrammes de classes, ce qui ne veut pas dire que la visibilité n'est pas définie dans le modèle. Par défaut, le niveau de visibilité est symbolisé par les caractères +, # et -, qui correspondent respectivement aux niveaux public, protégé et privé.

Associations

Les associations représentent des **relations structurelles entre classes d'objets**. Les associations se représentent en traçant une ligne entre les classes associées.

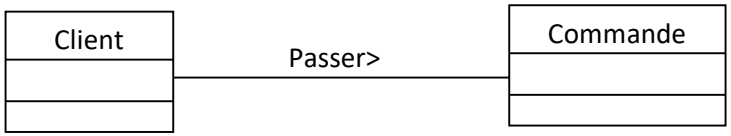


- **Arité des associations**

On appelle arité d'une association le nombre de classes qui participent à l'association. Il arrive en effet que l'information que l'on veut représenter nécessite la mise en relation de plus de deux classes. Ces associations n-aires peuvent se représenter au moyen d'un losange sur lequel arrivent les différents brins de l'association.

Identification d'une association

Les associations peuvent être nommées afin de faciliter la compréhension des modèles. Il est d'usage de nommer les associations par une forme verbale. On peut également préciser le sens de lecture par le biais d'un petit triangle dirigé vers la classe désignée par la forme verbale.



Rôle d'une association

- Les extrémités d'une association sont appelées rôles et peuvent porter un nom. Le rôle décrit comment une classe voit une autre classe au travers d'une association.
- Un rôle est nommé au moyen d'une forme nominale comme la montre la figure suivante.



On utilise rarement les deux modes de description d'association simultanément, mais on cherche celui le plus adapté à la situation à décrire.

Multiplicité des associations

Chaque rôle d'une association peut porter une multiplicité montrant combien d'objets de la classe considérée peuvent être liés à une instance de l'autre classe par l'association. La multiplicité est représentée sous la forme d'un couple de cardinalités.

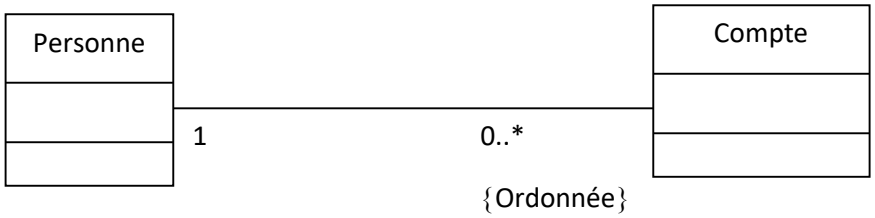
1..1	un et un seul
0..1	zéro ou un
0..* noté *	de Zéro à n
1..*	de un à n
n..m	de n à m

Valeurs de multicipité conventionnelles.

Les contraintes sur les associations

- Toutes sortes de contraintes peuvent être définies sur une relation ou sur un groupe de relations. Les contraintes se représentent dans les diagrammes par des expressions placées entre accolades.
- La contrainte **{ordonnée}** peut être placée sur le rôle pour spécifier qu'une relation d'ordre décrit les objets placés dans la collection ; dans ce cas, le modèle ne spécifie pas comment les éléments sont ordonnés, mais seulement que l'ordre doit être maintenu durant l'ajout et ou la suppression des objets par exemple.

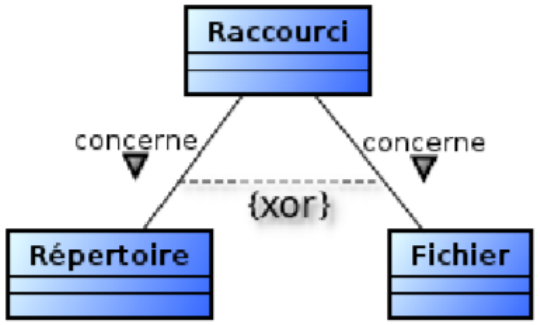
Dans l'exemple suivant, la contrainte **{ordonnée}** indique que la collection des comptes d'une personne est ordonnée.



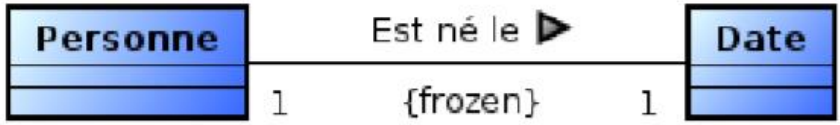
Contraintes et associations

Nous pouvons proposer des contraintes au niveau des associations.

Exemple 1 : Contrainte entre 2 associations.
En informatique, un raccourci peut concerner soit un répertoire soit un fichier (mais pas les 2 à la fois). Nous pouvons exprimer cela sous la forme d'une contrainte ***{xor}.***



Exemple 2 : Contrainte sur une association
Un objet de la classe ***Personne*** *est associé à un objet de la classe* ***Date****. La contrainte* ***{frozen}*** *indique que cette association ne peut plus être modifiée une fois instanciée.*



La relation de dépendance

La dépendance est la forme la plus faible de relation entre classes. Une dépendance entre deux classes signifie que l'une des deux utilise l'autre. Typiquement, il s'agit d'une relation transitoire, au sens où la première interagit brièvement avec la seconde sans conserver à terme de relation avec elle (liaison ponctuelle).

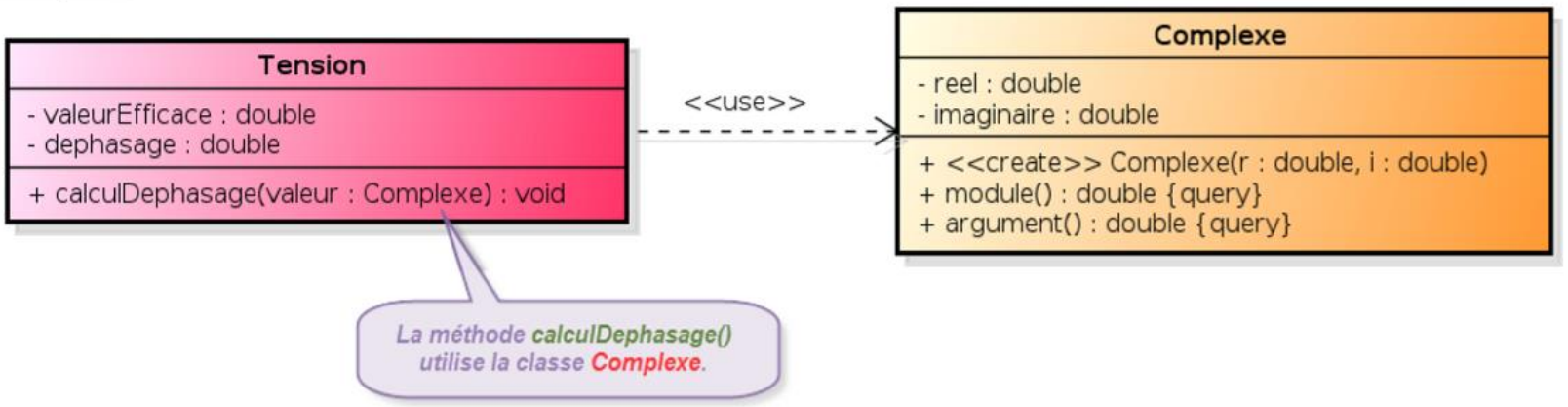
La relation de dépendance

Une dépendance peut s'interpréter comme une relation de type «utilise un». Elle est habituellement utilisée lorsqu'une classe utilise un objet d'une autre classe comme argument dans la signature d'une méthode ou alors lorsque l'objet de l'autre classe est créé à l'intérieur de la méthode. Dans les deux cas la durée de vie de l'objet est très courte, elle correspond à la durée d'exécution de la méthode.

La relation de dépendance

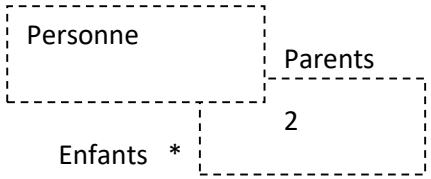
Notation : Elle est représentée par un trait discontinu orienté, reliant les deux classes. La dépendance est souvent stéréotypée (« use ») pour mieux expliciter le lien sémantique entre les éléments du modèle.

Exemple



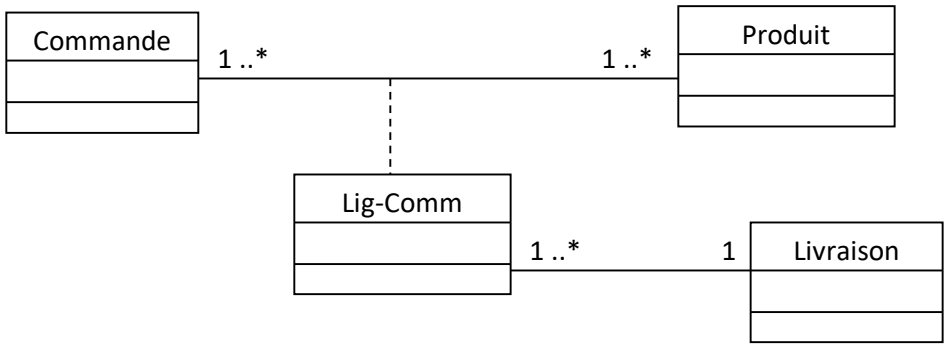
Association récursive

Les associations peuvent également relier une classe à elle-même, comme dans le cas des structures récursives. Le nommage des rôles prend à nouveau toute son importance pour distinguer les instances qui participent à la relation. L'exemple suivant montre la classe des personnes et la relation qui unie les parents et leurs enfants en vie.



Classe-association

Une association peut être représentée par une classe pour ajouter, par exemple, des attributs et des opérations dans l'association. Une classe de ce type, appelée parfois classe associative ou classe-association, est une classe comme les autres et peut à ce titre participer à d'autres relations dans le modèle.



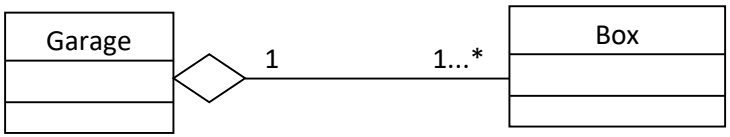
Agrégation

C'est une association binaire, orientée, de sémantique particulière, permettant de construire une hiérarchie entre classes d'objets composants et classes d'objets composites. Elle représente une association non symétrique dans laquelle une des extrémités joue un rôle prédominant par rapport à l'autre extrémité.

Les critères suivants impliquent une agrégation :

- Une classe fait partie d'une autre classe.
- Une action sur une classe implique une action sur une autre classe.
- Les objets d'une classe sont subordonnés aux objets d'une autre classe.

Les agrégations peuvent être multiples, comme les associations.

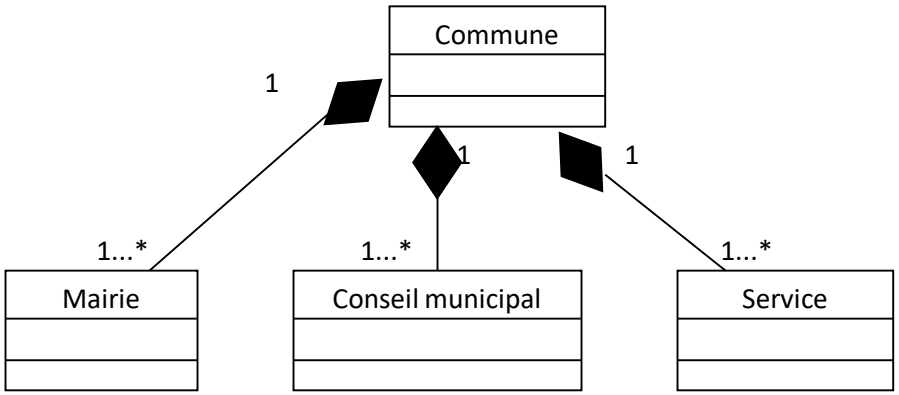


Composition

La composition est un cas particulier de l'agrégation dans laquelle la vie des composants est liée à celle de l'agrégat : la création, la modification et la destruction des divers composants sont de la responsabilité de l'objet agrégat.

Dans la composition, l'agrégat ne peut pas être multiple. La composition se représente par un losange noir.

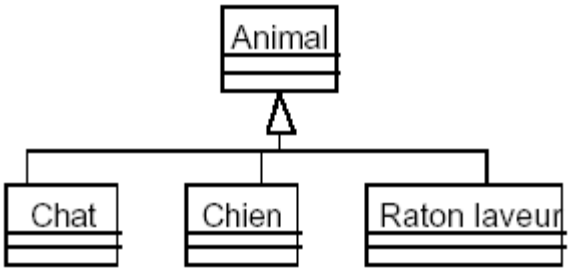
Une composition est une association contraignante : la suppression d'un objet agrégat entraîne la suppression des objets agrégés.



Généralisation/ Spécialisation

UML emploie le terme généralisation pour désigner la relation de classification entre un élément plus général et un élément plus spécifique. L'élément plus spécifique peut contenir des informations qui lui sont propres, à condition de rester totalement cohérent avec la description de l'élément plus général. La généralisation s'applique principalement aux classes, aux paquetages, aux cas d'utilisation et aux acteurs.

Dans le cas des classes, la relation de généralisation exprime le fait que les éléments d'une classe sont aussi décrits par une autre classe. La relation de généralisation signifie ***est un*** ou ***est une sorte de*** et ne porte aucun nom particulier.



Classes abstraites

Les classes abstraites ne sont pas instanciables directement car elles sont trop générales. Elles ne donnent pas naissance à des objets, mais servent de spécification plus générale – de type– pour manipuler les objets instances d'une (ou plusieurs) de leurs sous-classes. Une classe est désignée comme abstraite au moyen de la propriété booléenne **Abstraite** définie pour tous les éléments généralisables (les types, les paquetages et les stéréotypes). Par convention, le nom des classes abstraites est en italique.

Classes abstraites

Une classe concrète possède des instances. Elle constitue un modèle complet d'objet (tous les attributs et méthodes sont complètements décrits).

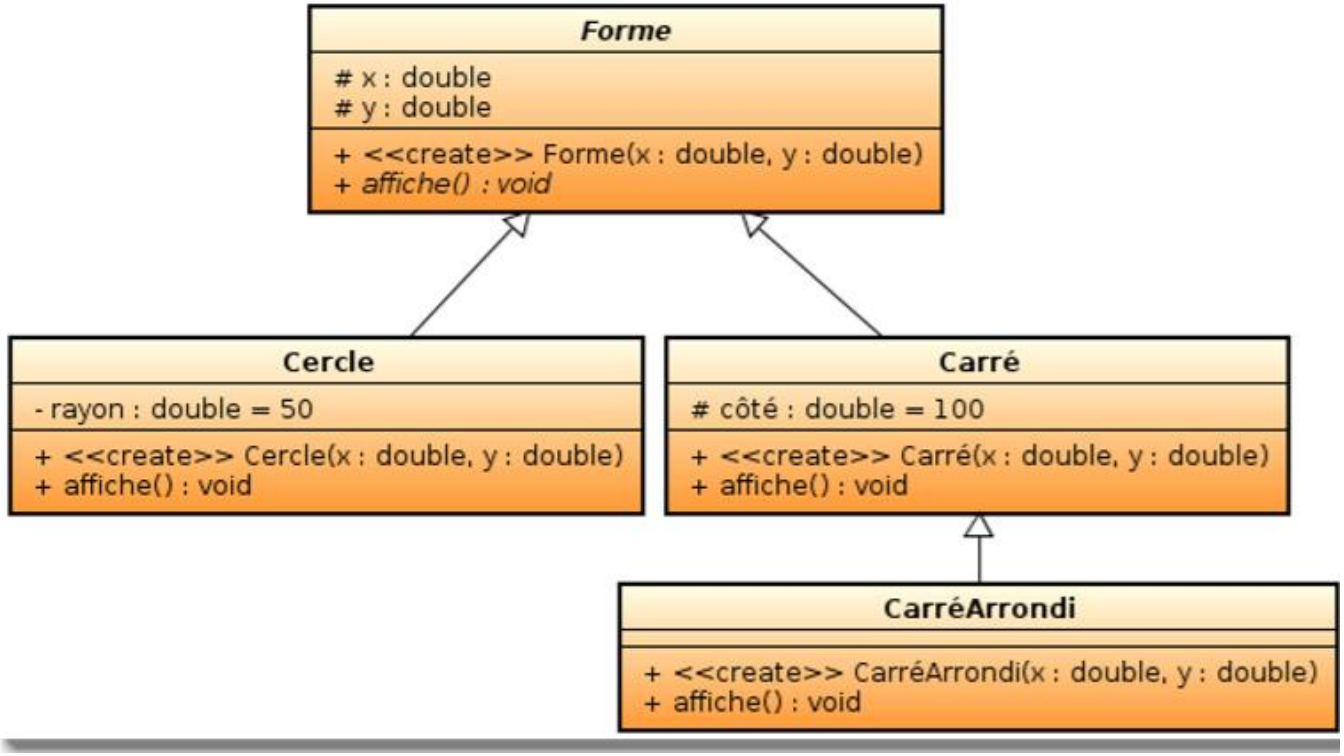
À l'opposé, une classe abstraite ne peut pas posséder d'instance directe car elle ne fournit pas une description complète. Elle a pour vocation de posséder des sous-classes concrètes et sert à factoriser des attributs et des méthodes à ses sous-classes.

Une classe abstraite possède généralement des méthodes communes aux sous-classes qui sont uniquement déclarées (sans codage interne). Une méthode introduite dans une classe avec sa seule signature et sans code est appelée une méthode abstraite.

En UML, une classe ou une méthode abstraite sont représentées avec une mise en italique du nom de la classe ou de la méthode.

Diagramme de classes

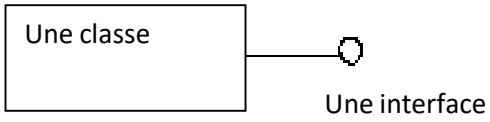
Toute classe possédant au moins une méthode abstraite est une classe abstraite. En effet, la seule présence d'une méthode incomplète (le code est absent) implique que la classe ne soit pas une description complète.



La classe Forme est abstraite puisqu'elle constituée de la méthode abstraite affiche(). Cette méthode est spécifiée abstraite puisqu'il est impossible de décrire un tracé particulier, ne connaissant pas la forme exacte. Ce n'est qu'avec une classe concrète comme Cercle que nous sommes capable de réaliser le tracé correspondant.

Interfaces

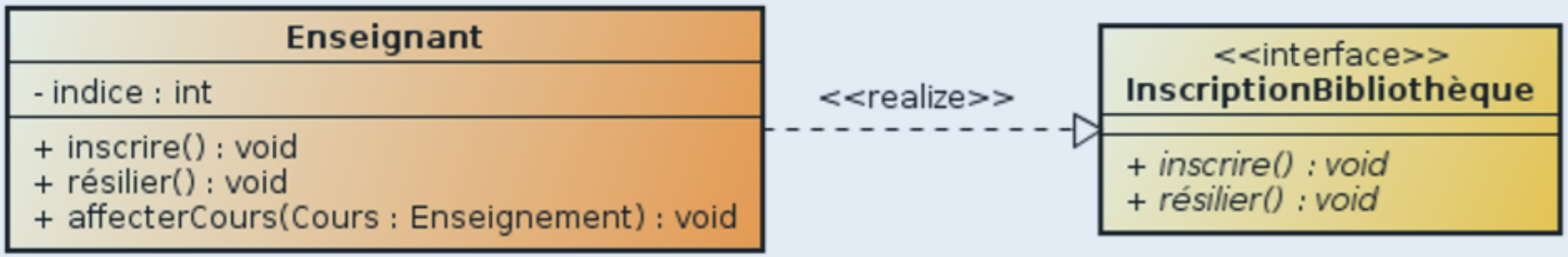
UML permet de représenter les interfaces des classes (qui décrivent le comportement visible de la classe) au moyen d'un cercle qui leur est associé ou par une classe avec le stéréotype **<<interface>>**.



Relation de réalisation et notion d'interfaces

- Une interface ne dispose que de méthodes publiques abstraites (on dit autrement, non définies). La généralisation peut être utilisée entre interfaces, par contre, une interface ne peut être instanciée (comme une classe abstraite).
- Comme l'interface n'a qu'un objectif de spécification, au moins un élément d'implémentation (une classe) doit lui être associée. La relation de réalisation permet justement de mettre en relation un élément de spécification avec son implémentation. Graphiquement, la réalisation se représente par un trait discontinu terminé par une flèche triangulaire parfois, mais pas nécessairement, stéréotypé « realize ».

Relation de réalisation et notion d'interfaces



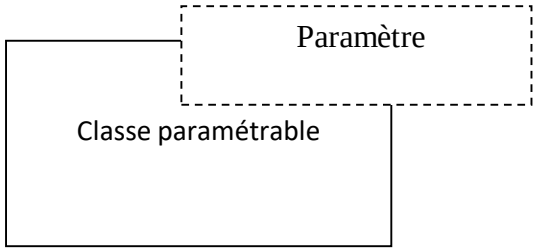
La relation de réalisation se représente en fait comme une relation d'héritage avec une ligne en pointillés. La raison de cette notation est que cette relation dénote une forme d'héritage partiel : héritage comportemental mais pas structurel. Une classe qui réalise (implémente) une interface est dans l'obligation de définir les méthodes décrites par l'interface (notion de contrat à respecter) comme lorsqu'une classe concrète redéfinie les méthodes abstraites héritées de sa classe parente abstraite.

Relation de réalisation et notion d'interfaces

Une classe peut très bien réaliser plusieurs interfaces, et une interface peut être réalisée par plusieurs classes (cas très fréquent). Enfin, c'est d'ailleurs l'intérêt des interfaces, une ou plusieurs classes vont utiliser les fonctionnalités justes nécessaires des classes qui réalisent une interface en passant par ladite interface.

Classes paramétrables

Les classes paramétrables sont des modèles de classes. Ces dernières doivent être instanciées avec des paramètres pour obtenir une classe concrète. UML représente une classe paramétrable de la façon suivante :



Classes utilitaires

Il est parfois utile de regrouper des éléments (les fonctions d'une bibliothèque mathématique par exemple) au sein d'un module, sans pour autant vouloir construire une classe complète. La classe utilitaire permet de représenter de tels modules et de les manipuler graphiquement comme les classes conventionnelles.

Les classes utilitaires ne peuvent pas être instanciées car elles ne sont pas des types de données. Il ne faut pas pour autant les confondre avec les classes abstraites qui ne peuvent pas être instanciées car elles sont des spécifications pures.

Le stéréotype <<**Utilitaire**>> spécialise les classes en classes utilitaires. UML propose également une représentation graphique alternative, avec une bordure grisée.



Représentation graphique d'une classe utilitaire.